Where qp keeps its data

Every table, file and cache qp writes or reads, and what survives a restart, a deploy and a lost box. Written from `quant-platform/chart/store.py`, `oplog.py`, `screener.py`, the CANSLIM modules and `tools/data/cache.py`.

Related: the screener tools' data, which qp reads but never opens directly (DATA-MODEL.md).

The rule

qp owns `~/.qp/` and `~/.qp-cache/`, and is the **only writer** of the O'Neil files in `data/` that the screener tools read. It reads the tools' registers **over HTTP** (`/api/warehouse/*` on each tool's port, or the archive's), never by opening their databases. Nothing qp keeps is in git except the seeds and the approvals.

The database: `~/.qp/platform.db`

SQLite. `QP_DB` moves it. It lives in the home folder on purpose: inside the repository, one routine git command could delete it.

Table	Columns	Holds
`strategies`	`id`, `name`, `data` (the whole strategy JSON), `created_at`, `updated_at`	every saved strategy (STRATEGIES.md)
`backtests`	`id`, `name`, `spec` (the run spec JSON, **including a frozen copy of the strategies it ran**), `status` (running, done, error), `progress` 0–1, `summary` (JSON), `error`, `created_at`, `updated_at`	every run (BACKTEST.md)
`backtest_trades`	`id`, `bt_id`, `date`, `symbol`, `side`, `entry_ts`, `exit_ts` (empty = still open at the day's end), `entry`, `exit`, `ret` (signed fraction), `reason` (SL, TP, trail, exit, eod, open), `ctx` (**the frozen register card**)	every trade of every run

Other files qp owns

Path	Holds	Notes
`~/.qp/strategies/*.json`	one JSON copy per strategy, written on every save	read back only when the database has no strategies at all
`quant-platform/chart/seeds/*.json`	the bundled strategies, in git	synced into the database at start (STRATEGIES.md)
`quant-platform/approvals/approvals.json`	the verification records of every primitive, in git	LIBRARY.md
`quant-platform/.default-feed`	the default feed chosen on the page	absent = `yahoo`
`quant-platform/chart/screener_sources.json`	which tools qp reads registers from	rebuilt from the tool registry; `SCREENER_SOURCES` overrides; `/api/screener/sources?reload=1` re-reads it
`quant-platform/chart/data/oplog-*.jsonl` (`QP_DATA_DIR`)	the operations log: what qp was asked, what it answered, how long it took	`GET /api/oplog`
`quant-platform/.env`	API keys for qp	written by the deploy from `data/keys.json`; never committed

Files qp writes for the screener tools

File in `data/`	Holds	Built by
`oneil-market.json`	the market model (M)	`chart/oneil.py`
`oneil-groups.json`	group ranks and rank-in-group (L)	`chart/groups.py`
`industry-map.json`	symbol → SIC industry and group (shared: the tools add TradingView labels only where SIC has no entry)	`chart/sic.py`, `chart/groups.py`
`oneil-13f.json`	institutional holders (I)	`chart/f13.py`
`oneil/` (`ONEIL_FUNDAMENTALS_DIR`)	EDGAR company facts per company, `_tickers.json`	`chart/edgar.py`

The data check flags any of these as stale after 36 hours (14 days for the industry map) (DATACHECK.md).

Caches (`~/.qp-cache/`, `QP_CACHE_DIR`)

Folder	Holds	Override
(top level)	bar frames, parquet, per feed	trimmed by size and free disk (DATA-FEEDS.md)
`grouped/`	Polygon grouped-daily whole-market frames, for the RS rating	—
`sic/`	SIC classification per company	`QP_SIC_CACHE`
`f13/`	Form 13F filings	`QP_13F_CACHE`
`canslim/`	CANSLIM letters as of each day	`QP_CANSLIM_CACHE`

Every cache can be rebuilt by fetching again.

In memory only

State	Lost on restart
the alert watcher’s state and suppression	it is off after a restart; a signal may alert once more when restarted
the running backtest’s thread	at start, qp marks every run left `running` as **interrupted** (“qp restarted while this ran. Run it again.”)

What survives what

Event	`~/.qp/platform.db` and JSON copies	`data/oneil-*`	caches	seeds, approvals
restart	kept	kept	kept	kept
deploy	kept (outside the repo)	kept	kept	updated from git
box lost	`platform.db` restored from the nightly backup; strategies also recoverable from backtest snapshots	rebuilt by the nightly job	re-fetched	from git